peek at the top of the heap. Then we can insert 3 to the heap, and we obtain [4, 3, 1, 2]. Now, if we want to keep the size of the heap at k elements, we need to remove one, the minimum: how do we know where it is, inside the heap? We only know where the maximum is (at the top of the max-heap), and so the search for the min could take up to linear time (even noting that the minimum will be in one of the leaves, there is unfortunately a linear number of them, n/D).

You can verify that, even by inserting the elements in a different order, we often find a similar situation.

The catch is that, when we want the k largest elements, at each step we are interested in understanding if the next number we evaluate is larger than the smallest of the k elements we already have. Hence, rather than a max-heap, we can use a min-heap bound to k elements where we store the largest elements found so far.

For each new element, we compare it to the top of the heap, and if the new one is smaller, we are sure it's not one of the k largest elements. If, instead, our new element is larger than the heap's top (i.e. the *smallest* of our k elements), then we extract the top from the heap, and then add our newly arrived. This way, updating the heap at each iteration costs us only constant time[16], instead of the linear time bound we had using a max-heap.

2.7.3 Coding it up

That's neat, right? And simple to implement. Since code is worth a thousand words, let's see our heap in action in listing